

JavaScript Learning Notes

Everything covered so far — concepts, explanations, and your practice code

Covered: Variables → Data Types → Operators → if/else → Functions → return → Arrays → Loops → forEach → map → filter → reduce → chaining → Objects → Object methods

1. Variables & Basic JavaScript

JavaScript can be practiced directly in the browser using Chrome DevTools → Console.

```
let name = "Nitin";
console.log(name);

let age = 25;
age = 26;

const city = "Panchkula";
```

2. Data Types

```
// String
let name = "Nitin";

// Number
let age = 25;

// Boolean
let isLoggedIn = true;

// Undefined
let x;

// Null
let user = null;

// Object
let person = { name: "Nitin", age: 25 };

// Array
let fruits = ["Apple", "Mango", "Banana"];
```

3. Operators & if/else

```
let a = 10;
let b = 5;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
```

Comparison: === means strict equality. >= includes the boundary value.

```
let age = 25;

if (age >= 18) {
  document.write("Adult");
} else {
  document.write("Minor");
}
```

4. Functions, Parameters & return

```
function greet() {
  console.log("Hello Nitin");
}

greet();
```

```
function add(a, b) {
    return a + b;
}

console.log(add(10, 20));
```

Your practice:

```
function multiply(a, b){
    return a * b;
}

console.log(multiply(2, 4));
console.log(multiply(244, 784));
console.log(multiply(23, 44));

function square(number){
    return number ** 2;
}

console.log(square(5));

function checkAge(age){
    if(age >= 18){
        return "Adult";
    }else{
        return "Minor";
    }
}

console.log(checkAge(5));
```

5. Arrays

```
let person = ["Nitin", "Amit", "Rahul", "Vikas"];

console.log(person[2]);
console.log(person.length);

person.push("Sahil");
console.log(person);
```

Important: Array indexes start at 0. push() adds an item and returns the new length.

```
for (let x = 0; x < person.length; x++) {
    console.log(person[x]);
}

for (let name of person) {
    console.log(name);
}
```

6. Arrays + Even/Odd + Functions

```
let numbers = [10, 25, 30, 45, 50, 63];

console.log(numbers[2]);

numbers.push(100);
console.log(numbers);

for (let number of numbers) {
    console.log(number);
}

for (let number of numbers) {
    if (number % 2 === 0) {
        console.log(number);
    }
}

function printEven(numbers) {
    for (let number of numbers) {
        if (number % 2 === 0) {
```

```

        console.log(number);
    }
}

printEven(numbers);

```

Remember: numbers is the whole array; number is one individual item inside the loop.

7. forEach()

```

let numbers = [10, 20, 31, 44, 55, 60];

numbers.forEach(function(number){
    console.log(number);
});

numbers.forEach(function(number){
    if(number % 2 === 0){
        console.log(number);
    }
});

numbers.forEach(function(number){
    console.log(number * 2);
});

function printEven(numbers){
    numbers.forEach(function(number){
        if(number % 2 === 0){
            console.log(number);
        }
    });
}

printEven(numbers);

```

8. map()

map() transforms every item and returns a **new array**. In map(), use return when you want the transformed array.

```

let numbers = [10, 20, 30, 40, 50];

let result = numbers.map(function(number){
    return number * 2;
});

console.log(result);

```

Your practice examples:

```

numbers.map(function(number){
    console.log(number * 2);
});

numbers.map(function(number){
    console.log(number ** 2);
});

let names = ["Nitin", "Amit", "Rahul"];

names.map(function(name){
    console.log("Hello " + name);
});

let prices = [100, 200, 300, 400];

let result = prices.map(function(price){
    return price + (price * 10 / 100);
});

console.log(result);

let priceText = prices.map(function(price){

```

```

        return "■" + price;
    });

    console.log(priceText);

    let users = ["Nitin", "Amit", "Rahul"];

    let userObjects = users.map(user => {
        return { name: user };
    });

    console.log(userObjects);

```

9. filter()

`filter()` selects items that satisfy a condition and returns a **new array**.

```

let numbers = [10, 15, 20, 25, 30, 35, 40];

let result = numbers.filter(number => {
    return number % 2 === 0;
});

console.log(result);

let greater = numbers.filter(number => {
    return number > 20;
});

console.log(greater);

let users = [
    {name: "Nitin", age: 25},
    {name: "Amit", age: 17},
    {name: "Rahul", age: 30},
    {name: "Vikas", age: 16}
];

let adults = users.filter(user => {
    return user.age >= 18;
});

console.log(adults);

```

10. reduce()

`reduce()` combines array values into one final value. Common initial values: 0 for addition and 1 for multiplication.

```

let numbers = [10, 20, 30, 40, 50];

let result = numbers.reduce(function(sum, number){
    return sum + number;
}, 0);

console.log(result);

let multiply = numbers.reduce(function(product, number){
    return product * number;
}, 1);

console.log(multiply);

let prices = [100, 200, 300, 400];

let total = prices.reduce(function(total, price){
    return total + price;
}, 0);

console.log(total);

let users = [
    {name: "Nitin", age: 25},
    {name: "Amit", age: 17},

```

```

    {name: "Rahul", age: 30}
  ];

  let ageTotal = users.reduce(function(ageTotal, user){
    return ageTotal + user.age;
  }, 0);

  console.log(ageTotal);

```

11. filter → map → reduce

```

let numbers = [10, 15, 20, 25, 30, 35, 40];

let result = numbers.filter(function(number){
  return number % 2 === 0;
});

console.log(result);

let mapdata = result.map(function(resl){
  return resl * 2;
});

console.log(mapdata);

let reducedata = mapdata.reduce(function(sum, reddd){
  return sum + reddd;
}, 0);

console.log(reducedata);

```

Result: [10, 20, 30, 40] → [20, 40, 60, 80] → 200.

```

let result = numbers
  .filter(number => number % 2 === 0)
  .map(number => number * 2)
  .reduce((sum, number) => sum + number, 0);

console.log(result);

```

Memory trick: map = change/transform | filter = select/choose | reduce = combine

12. Objects

```

let user = {
  name: "Nitin",
  age: 25,
  city: "Panchkula"
};

console.log(user.name);
console.log(user["name"]);

user.age = 26;

user.email = "test123@gmail.com";

delete user.city;

console.log(user);

```

You practiced accessing, updating, adding, and deleting object properties.

13. Nested Objects & Array of Objects

```

let user = {
  name: "Nitin",
  age: 25,
  address: {
    city: "Panchkula",
    state: "Haryana"
  },
  greet: function(){

```

```

        console.log("Hello Nitin");
    }
};

console.log(user.address.city);
user.greet();

let users = [
    { name: "Nitin", age: 25 },
    { name: "Amit", age: 30 },
    { name: "Rahul", age: 22 }
];

console.log(users[0]);
console.log(users[0].name);
console.log(users[1].age);

```

14. Object.keys(), values(), entries()

```

let user = {
    name: "Nitin",
    age: 26,
    email: "test123@gmail.com"
};

console.log(Object.keys(user));
// ["name", "age", "email"]

console.log(Object.values(user));
// ["Nitin", 26, "test123@gmail.com"]

console.log(Object.entries(user));
// [
//   ["name", "Nitin"],
//   ["age", 26],
//   ["email", "test123@gmail.com"]
// ]

Object.keys(user).forEach(function(key){
    console.log(key);
});

Object.values(user).forEach(function(value){
    console.log(value);
});

Object.entries(user).forEach(function([key, value]){
    console.log(key, value);
});

```

Next topic: Destructuring. The [key, value] part in the last example is an introduction to destructuring.

15. Progress

Variables / data types / operators	Done
if / else	Done
Functions / parameters / return	Done
Arrays / loops / forEach	Done
map / filter / reduce	Done
Method chaining	Done
Objects / nested objects	Done
Array of objects	Done
Object.keys / values / entries	Done

